



Học
Luyện tập
Code
Làm chủ

CẨM NANG VIẾT TAY

Code
hôm nay
Giỏi hơn
ngày mai

NHẬP MÔN JAVA

100%
Ghi chú
viết tay
★

```
class Hello {  
    public static void  
    main(String[] args) {  
        System.out.println  
        ("Hello Java");  
    }  
}
```



- ✓ Kiến thức cơ bản
- ✓ Cú pháp
- ✓ Khái niệm OOP
- ✓ Bài tập thực hành
- ✓ Sẵn sàng phỏng vấn

- ✦ Đơn giản
- ✦ Có cấu trúc
- ✦ Dễ hiểu
- ✦ Hoàn hảo cho người mới bắt đầu

```
class Code { void run() { keepCoding(); } }
```





Hiểu rõ **kiến thức cơ bản**.
Luyện tập **đều đặn**.
Code **tự tin**.
Làm chủ Java.

Bước nhỏ
mỗi ngày.
Kết quả lớn
một ngày.



CẨM NANG NHẬP MÔN JAVA

Cẩm nang
viết tay
★

1. JAVA LÀ GÌ?

Java là một ngôn ngữ lập trình **high-level**, **class-based**, **object-oriented** được phát triển bởi **Sun Microsystems** (hiện thuộc sở hữu của **Oracle**). Java được phát hành chính thức vào năm **1995**.

Đặc điểm chính của Java

- ① **Simple** - Dễ học và dễ hiểu.
- ② **Object-Oriented** - Dựa trên class và object.
- ③ **Platform Independent** - Viết một lần, chạy mọi nơi (WORA) nhờ JVM.
- ④ **Secure** - Không có pointer, quản lý bộ nhớ chặt chẽ.
- ⑤ **Robust** - Quản lý bộ nhớ tốt và xử lý exception hiệu quả.
- ⑥ **Multithreaded** - Hỗ trợ lập trình concurrent.
- ⑦ **Portable** - Cùng một chương trình chạy được trên mọi nền tảng nhờ JVM.
- ⑧ **High Performance** - Trình biên dịch JIT (Just In Time) giúp cải thiện hiệu năng.
- ⑨ **Distributed** - Hỗ trợ networking sẵn có.
- ⑩ **Dynamic** - Có thể thích ứng với môi trường thay đổi.

Vì sao Java phổ biến?

- ⚡ Cú pháp dễ dùng
- 🌐 Độc lập nền tảng (platform independent)
- 🔥 Bảo mật tốt
- ✅ Dùng trong web, mobile, desktop, cloud, AI/ML, Big Data, v.v.

Phương châm của Java

*"Write Once,
Run Anywhere"*

Logo Java



Java

2. LỊCH SỬ JAVA

- 1991 → Project Green được khởi động bởi James Gosling
- 1995 → Java được phát hành chính thức bởi Sun Microsystems
- 1996 → Java 1.0
- 1997 → Java 1.1 (Inner class, JDBC, v.v.)
- 1998 → Java 1.2 (Collection Framework, Swing, v.v.)
- 2004 → Java 5 (Generics, Annotations, v.v.)
- 2014 → Oracle phát hành Java 8
- 2023+ → Các phiên bản LTS mới nhất và các bản cập nhật...

3. ỨNG DỤNG CỦA JAVA

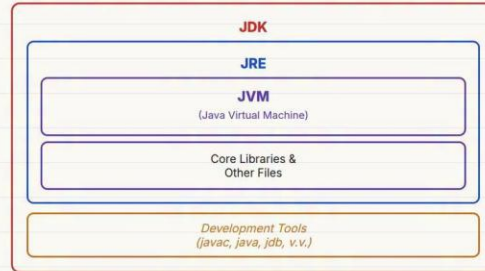
- 🚀 Phát triển ứng dụng Android
- 🚀 Ứng dụng Web
- 🚀 Ứng dụng Enterprise
- 🚀 Ứng dụng Desktop
- 🚀 Công nghệ Big Data (Hadoop)
- 🚀 Ứng dụng Cloud
- 🚀 Game
- 🚀 Hệ thống nhúng (Embedded Systems), v.v.

💡 **Ghi nhớ:** Java không chỉ là một ngôn ngữ lập trình, mà còn là một **platform** để xây dựng các giải pháp hoàn chỉnh. 😊

4. JDK, JRE VÀ JVM

- **JDK** (Java Development Kit) - Cung cấp công cụ để phát triển ứng dụng Java. Bao gồm JRE + compiler (javac), debugger, và các công cụ khác.
- **JRE** (Java Runtime Environment) - Cung cấp thư viện và các file cần thiết để chạy chương trình Java. Bao gồm JVM + core class.
- **JVM** (Java Virtual Machine) - Là một engine chạy Java bytecode. Cung cấp khả năng độc lập nền tảng. Mỗi nền tảng có một JVM riêng.

Chúng hoạt động cùng nhau như thế nào?



java Source Code

↓ javac

.class Bytecode

↓ JVM (Interpreter)

Machine Code

↓

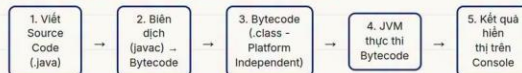
Output

5. CẤU TRÚC CHƯƠNG TRÌNH JAVA

```
// Đây là comment một dòng
/*
    Đây là comment
    nhiều dòng
*/
public class HelloWorld {
    public static void main(String[] args) {
        // Code của bạn ở đây
        System.out.println("Hello, World!");
    }
}
```

- public class HelloWorld → Tên class phải trùng với tên file.
- public static void main(String[] args) → Điểm bắt đầu (entry point) của mọi chương trình Java.
- System.out.println() → In kết quả ra console.

7. QUY TRÌNH BIÊN DỊCH VÀ THỰC THI



6. CÁC LOẠI COMMENT TRONG JAVA

Loại	Cú pháp / Ví dụ
Single Line	// Đây là comment
Multi Line	/* Đây là comment nhiều dòng */
Documentation	/** * Đây là comment * documentation */

8. CHƯƠNG TRÌNH JAVA ĐẦU TIÊN

```
public class MyFirstProgram {
    public static void main(String[] args) {
        System.out.println("Welcome to Java!");
    }
}
```



Mẹo nhanh: Viết code → Biên dịch → Chạy → Debug → Cải thiện. Luyện tập chính là chìa khóa để trở thành lập trình viên giỏi hơn.



9. BIẾN TRONG JAVA

Một **biến (variable)** là một vị trí được đặt tên trong bộ nhớ, dùng để lưu trữ dữ liệu có thể thay đổi trong quá trình chương trình chạy.

Cú pháp:

```
data_type variable_name = value;
```

Ví dụ:

```
int age = 25;
double price = 99.99;
char grade = 'A';
String name = "Nikhil";
boolean isJavaFun = true;
```

10. KIỂU DỮ LIỆU TRONG JAVA

(A) Kiểu dữ liệu Primitive

Kiểu	Kích thước	Phạm vi / Giá trị mặc định	Ví dụ
byte	1 byte	-128 đến 127 / 0	byte b=10;
short	2 bytes	-32,768 đến 32,767 / 0	short s=100;
int	4 bytes	-2 ³¹ đến 2 ³¹ -1 / 0	int i=50;
long	8 bytes	-2 ⁶³ đến 2 ⁶³ -1 / 0L	long l=100000L;
float	4 bytes	~6-7 chữ số / 0.0f	float f=5.5f;
double	8 bytes	~15 chữ số / 0.0d	double d=10.25;
char	2 bytes	\u0000 đến \uffff	char c='A';
boolean	1 bit	true / false	boolean b=true;

(B) Kiểu dữ liệu Non-Primitive (Reference)

String, Arrays, Classes, Objects, Interfaces, v.v.

11. Ép KIỂU (TYPE CASTING)

(A) Chuyển đổi ngầm định (Widening)

Được compiler tự động thực hiện.

```
int i = 10;
double d = i; // int sang double
```

(B) Chuyển đổi tường minh (Narrowing)

Được lập trình viên thực hiện thủ công.

```
double d = 10.5;
int i = (int)d; // double sang int
```

13. ĐỊNH DANH (IDENTIFIER) TRONG JAVA

Một **identifier** là tên được đặt cho class, biến, phương thức, object, v.v.

Quy tắc:

- Có thể chứa chữ cái (A-Z, a-z), số (0-9), _ và \$.
- Không được bắt đầu bằng số.
- Không có khoảng trắng hoặc ký tự đặc biệt (@, %, #, v.v.)
- Phân biệt hoa thường (myVar và myvar là khác nhau).

Ví dụ:

- ✓ Hợp lệ: myVar, _name, \$value, total1, age2
- ✗ Không hợp lệ: 2name, my var, @value, #age

12. LITERAL TRONG JAVA

Loại	Ví dụ
Integer Literal	100, -25, 0
Floating Literal	10.5, -3.14, 2.0e4
Character Literal	'A', 'z', '7', '\$'
String Literal	"Hello", "Java Programming"
Boolean Literal	true, false
Null Literal	null

Lưu ý

Quy tắc đặt tên biến

- Phải bắt đầu bằng chữ cái, dấu gạch dưới (_) hoặc \$
- Không được bắt đầu bằng số.
- Phân biệt hoa thường.
- Không dùng ký tự đặc biệt, trừ _ và \$. 💡

14. TỪ KHÓA (KEYWORD) TRONG JAVA

Keyword là những từ dành riêng và không thể dùng làm identifier.

```
abstract assert boolean break byte case catch char class const
continue default do double else enum extends final finally float for
if implements import instanceof int interface long native new package
private protected public return short static strictfp super switch
synchronized this throw throws transient try void volatile while
```

Lưu ý: const và goto được để dành (reserved) nhưng không được sử dụng.



Ghi nhớ: Nền tảng vững chắc tạo nên lập trình viên giỏi. Practice → Understand → Implement → Excel 😊

15. TOÁN TỬ (OPERATOR) TRONG JAVA

Operator dùng để thực hiện các phép toán trên biến và giá trị.

Loại	Toán tử	Ví dụ
Arithmetic	+ - * / %	a + b, a % b
Unary	+ - ++ -- !	++a, b--, !flag
Relational	= < > <= >=	a >= b
Logical	&& !	(a>b) && (c<d)
Assignment	= += -= *= /= %=	a += 5
Bitwise	& ^ ~ << >> >>>	a & b, a << 2
Ternary	? :	(age >= 18) ? "Yes" : "No"

16. ĐỘ ƯU TIÊN TOÁN TỬ

Toán tử được tính theo độ ưu tiên (precedence). Toán tử có độ ưu tiên cao hơn được tính trước.

Độ ưu tiên	Toán tử
1 (Cao nhất)	() , [] , . , ++ , --
2	*, / , %
3	+, -
4	<<, >>, >>>
5	<, <=, >, >=, instanceof
6	=, <=, >=, <, >, <=, >=
7	&
8	^
9	
10	&&
11	
12 (Thấp nhất)	=, +=, -=, *=, /=, %=

17. CHUYỂN ĐỔI KIỂU (CASTING)**(A) Ngầm định (Widening)**

Được compiler tự động thực hiện. An toàn, không mất dữ liệu.

byte → short → int → long → float → double

char → int → long → float → double

Ví dụ:

```
int i = 100;
double d = i; // ngầm định

double x = 10.5;
int y = (int) x; // tường minh
```

(B) Tường minh (Narrowing)

Do lập trình viên thực hiện. Có thể mất dữ liệu (rủi ro).

18. NHẬP DỮ LIỆU (INPUT) TRONG JAVA

Dùng class Scanner (java.util.Scanner)

```
import java.util.Scanner;
public class InputExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter name: ");
        String name = sc.nextLine(); // String
        System.out.print("Enter age: ");
        int age = sc.nextInt(); // int
        sc.close();
    }
}
```

19. XUẤT DỮ LIỆU (OUTPUT) TRONG JAVA

Dùng System.out

System.out.print()	In không xuống dòng
System.out.println()	In có xuống dòng
System.out.printf()	In có định dạng
System.out.format()	Giống printf()
System.out.write()	In character stream

20. ESCAPE SEQUENCE TRONG JAVA

Dùng trong string để biểu diễn ký tự đặc biệt.

Escape	Ý nghĩa	Escape	Ý nghĩa
\n	Xuống dòng mới	\"	Dấu nhảy kép
\t	Tab	\'	Dấu nhảy đơn
\r	Carriage return	\\	Dấu gạch chéo ngược
\b	Backspace	\uXXXX	Ký tự Unicode
\f	Form feed		

```
System.out.println("Hello\nJava"); // New Line
System.out.println("Name:\tNikhil"); // Tab
```



Ghi nhớ: Toán tử và cú pháp là nền tảng của mọi biểu thức Java. Luyện tập thật nhiều để thành thạo. 😊

21. CÂU LỆNH ĐIỀU KHIỂN (CONTROL STATEMENT)

Control statement dùng để điều khiển luồng thực thi của chương trình.

Selection / Decision	Iteration / Looping	Jump Statement
if if-else else-if ladder nested if switch	for while do-while enhanced for (for-each loop)	break continue return

Ví dụ:

```
if (age >= 18) {  
    System.out.println("Adult");  
} else {  
    System.out.println("Minor");  
}  
  
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}  
  
while (n > 0) { n--; }
```

23. MẢNG (ARRAY) TRONG JAVA

Array là tập hợp các phần tử cùng kiểu, được lưu trữ tại các vị trí bộ nhớ liên tiếp.

Khai báo:

```
dataType[] arrayName = new dataType[size];
```

Ví dụ:

```
int[] marks = new int[5]; // khai báo  
marks[0] = 85; marks[1] = 90;  
marks[2] = 78; ... marks[4] = 88;
```

Index	0	1	2	3	4
marks	85	90	78	92	88

Lưu ý: Index luôn bắt đầu từ 0.

25. PHƯƠNG THỨC (METHOD) TRONG JAVA

Method là một khối code thực hiện một tác vụ cụ thể.

```
returnType methodName(parameters) {  
    // code thực thi  
    return value; // nếu returnType khác void  
}
```

22. VÒNG LẶP (LOOP) TRONG JAVA

Loop	Khi nào dùng	Cú pháp
for	Biết trước số lần lặp.	for (init; cond; update) { }
while	Chưa biết trước số lần lặp (dựa trên điều kiện).	while (condition) { }
do-while	Thực thi ít nhất 1 lần, sau đó mới kiểm tra điều kiện.	do { } while (cond);
enhanced for	Duyệt qua array hoặc collection.	for (type v: arr) { }

Ví dụ (for-each):

```
int[] arr = {10, 20, 30, 40};  
for (int num : arr) {  
    System.out.println(num);  
}
```

24. CHUỖI (STRING) TRONG JAVA

String là một chuỗi ký tự. Trong Java, String là một object của class String.

```
String name = "Nikhil";
```

Các phương thức thường dùng:

length()	Trả về độ dài chuỗi
charAt(i)	Trả về ký tự tại index i
substring(a,b)	Trả về chuỗi con (a đến b-1)
toUpperCase()	Chuyển thành chữ hoa
equals()	So sánh hai chuỗi

26. NẠP CHỒNG PHƯƠNG THỨC (METHOD OVERLOADING)

Method overloading là việc có nhiều method cùng tên nhưng khác tham số, trong cùng một class.

```
int add(int a, int b) { return a + b; }  
double add(double a, double b) { return a+b; }  
int add(int a, int b, int c) { return a+b+c; }  
// Compile-time polymorphism
```



Ghi nhớ: Vòng lặp, mảng và chuỗi là công cụ cốt lõi để xử lý dữ liệu trong Java.



27. KHÁI NIỆM OOP (OOPs CONCEPTS)

Java là ngôn ngữ lập trình **Object-Oriented**. OOP giúp code đáng tin cậy, linh hoạt và dễ bảo trì hơn.

Bốn trụ cột của OOP

1. **Encapsulation** - Đóng gói dữ liệu và code thành một khối duy nhất.
2. **Abstraction** - Ẩn chi tiết bên trong, chỉ hiển thị tính năng cần thiết.
3. **Inheritance** - Kế thừa thuộc tính của một class vào class khác.
4. **Polymorphism** - Khả năng tồn tại dưới nhiều hình thức khác nhau.

28. CLASS VÀ OBJECT

Class - Bản thiết kế (blueprint) hoặc khuôn mẫu để tạo object.

Object - Một thực thể (instance) của class.

```
// Ví dụ
class Car {
    String brand;           // biến
    void start() {          // method
        System.out.println("Car Started");
    }
}
Car obj = new Car(); // tạo object
obj.brand = "Tesla";
obj.start();
```

💡 **Suy nghĩ:** Class là bản kế hoạch, Object là thứ có thật.

29. ACCESS MODIFIER

Access modifier kiểm soát phạm vi truy cập của thành viên trong class.

Modifier	Trong Class	Trong Package	Package khác (Subclass)	Package khác
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

30. CONSTRUCTOR TRONG JAVA

Constructor là một method đặc biệt dùng để khởi tạo object. Tên trùng với tên class và không có kiểu trả về.

Các loại Constructor:

- **Default Constructor** - Không có tham số.
- **Parameterized Constructor** - Có tham số.

```
class Student {
    int id; String name;
    // Default Constructor
    Student() { id = 0; name = "Unknown"; }
    // Parameterized Constructor
    Student(int id, String name) {
        this.id = id; this.name = name;
    }
}
Student s1 = new Student(); // default
Student s2 = new Student(101, "Nikhil"); // parameterized
```

Bảng tổng hợp - Các chủ đề đã học

Chủ đề	Điểm chính	Chủ đề	Điểm chính
What is Java?	High-level, class-based, object-oriented, độc lập nền tảng, bảo mật, robust.	Arrays & Strings	Array lưu nhiều giá trị. String là chuỗi ký tự.
JDK, JRE, JVM	JDK = Development Kit, JRE = Runtime Environment, JVM = Execution Engine.	Methods	Khởi code tái sử dụng, có hoặc không có tham số.
Variables & Data Types	Lưu trữ dữ liệu. Kiểu: primitive & non-primitive.	OOPs Concepts	Encapsulation, Abstraction, Inheritance, Polymorphism.
Operators	Arithmetic, Relational, Logical, Assignment, Bitwise, Ternary.	Class & Object	Class là bản thiết kế, object là thực thể.

31. TỪ KHÓA THIS

this là biến tham chiếu, trỏ đến object hiện tại của class.

- Dùng để tham chiếu biến instance của class hiện tại.
- Dùng để gọi method của class hiện tại.
- Dùng để gọi constructor của class hiện tại.
- Được truyền như một tham số trong lời gọi method.

Ví dụ:

```
class Student {
    int id; String name;
    Student(int id, String name) {
        this.id = id;    // tham chiếu biến instance
        this.name = name;
    }
    void show() {
        System.out.println(this.id + " " + this.name);
    }
}
```

32. TỪ KHÓA STATIC

static dùng để quản lý bộ nhớ. Nó thuộc về class chứ không thuộc về instance nào.

Đặc điểm	Static	Non-Static (Instance)
Thuộc về	Class	Object
Bộ nhớ	Cấp phát 1 lần	Cấp phát mỗi khi tạo object
Truy cập	Qua tên class	Qua tham chiếu object
Có thể truy cập	Chỉ thành viên static	Cả static và non-static

Ví dụ:

```
class Demo {
    static int count = 0; // biến static
    int id;               // biến instance
    static void printCount() { // method static
        System.out.println(count);
    }
}
```

33. TỪ KHÓA FINAL

final dùng để hạn chế thay đổi. Có thể áp dụng cho biến, method hoặc class.

Áp dụng cho	Ý nghĩa	Ví dụ
final variable	Giá trị không đổi được	final int MAX=100;
final method	Không thể override	final void show(){}
final class	Không thể kế thừa	final class A{}

35. ĐA HÌNH (POLYMORPHISM)

Polymorphism nghĩa là "nhiều hình thức". Cho phép một interface được dùng cho nhiều kiểu khác nhau.

Các loại: 1. Compile-time (Method Overloading) 2. Run-time (Method Overriding)

```
// 1) Method Overloading (Compile-time)
class MathDemo {
    int add(int a,int b){return a+b;}
    double add(double a,double b){return a+b;}
}

// 2) Method Overriding (Run-time)
class Animal { void sound(){System.out.println("Animal");} }
class Dog extends Animal { void sound(){System.out.println("Bark");} }
```

34. KẾ THỪA (INHERITANCE)

Inheritance cho phép một class (con) kế thừa field và method từ một class khác (cha).

Các loại Inheritance

- Single** - Một con, một cha
- Multilevel** - Con kế thừa từ một con khác
- Hierarchical** - Nhiều con, một cha
- Hybrid** - Kết hợp các loại trên (qua interface)

Ví dụ (Single Inheritance):

```
class Animal {
    void eat(){System.out.println("Eating...");}
}
class Dog extends Animal {
    void bark(){System.out.println("Barking...");}
}
```

36. TRỪU TƯỢNG (ABSTRACTION)

Abstraction là việc ẩn chi tiết cài đặt, chỉ hiển thị chức năng. Đạt được thông qua abstract class và interface.

Abstract Class

- Có thể có method abstract và non-abstract
- Có thể có constructor

Interface

- Tất cả method mặc định là abstract
- Biến mặc định là public static final

🔥 **Ghi nhớ:** this, static và final là những từ khóa nền tảng - hiểu rõ chúng giúp code Java "sạch" hơn rất nhiều. 😊

37. XỬ LÝ EXCEPTION

Exception là một sự kiện ngoài ý muốn làm gián đoạn luồng chạy bình thường của chương trình.

Từ khóa: try, catch, finally, throw, throws

Từ khóa	Mục đích
try	Bao code có thể ném exception
catch	Bắt và xử lý exception
finally	Luôn thực thi (dù có exception hay không)
throw	Ném exception thủ công
throws	Khai báo exception (trong method signature)

Vi dụ:

```
public class ExDemo {
    public static void main(String[] args) {
        try {
            int a=10, b=0;
            int c = a/b; // có thể ném ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        } finally {
            System.out.println("Finally block executed");
        }
    }
}
```

38. CÁC EXCEPTION THƯỜNG GẶP

Exception	Nguyên nhân
ArithmeticException	Chia cho 0, phép tính không hợp lệ
ArrayIndexOutOfBoundsException...	Truy cập index không hợp lệ
NullPointerException	Truy cập qua tham chiếu null
NumberFormatException	Chuyển String không hợp lệ sang số
IOException	Thao tác Input/Output thất bại
IllegalArgumentException	Truyền tham số không hợp lệ

39. PHÂN LOẠI EXCEPTION

1. Checked Exception

- Kiểm tra lúc compile-time
- Phải xử lý hoặc khai báo
- VD: IOException

2. Unchecked Exception

- Kiểm tra lúc runtime
- Không bắt buộc xử lý
- VD: ArithmeticException

40. COLLECTIONS FRAMEWORK

Java Collections Framework cung cấp cách lưu trữ và thao tác với nhóm các object.



List vs Set

List	Set
Có thứ tự, cho phép trùng lặp	Không có thứ tự, không trùng lặp
VD: ArrayList	VD: HashSet

Vi dụ (ArrayList):

```
ArrayList<String> list = new ArrayList<>();
list.add("Java"); list.add("Hello");
System.out.println(list.get(1)); // Hello
```

41. INTERFACE DÙNG CHUNG

Comparator và Comparable dùng để sắp xếp object.

(A) Comparable (sắp xếp mặc định)

```
class Student implements Comparable<Student> {
    int roll; String name;
    public int compareTo(Student s) {
        return this.roll - s.roll; // tăng dần
    }
}
```

(B) Comparator (sắp xếp tùy chỉnh)

```
class NameComparator implements Comparator<Student> {
    public int compare(Student s1, Student s2) {
        return s1.name.compareTo(s2.name);
    }
}
```

42. ĐIỂM QUAN TRỌNG CẦN NHỚ

- ✓ Java độc lập nền tảng (Write Once, Run Anywhere).
- ✓ Mọi thứ trong Java được truyền theo giá trị (trừ tham chiếu object).
- ✓ JVM chuyển file .class thành machine code.
- ✓ Tuân theo nguyên tắc OOP để code tốt hơn.
- ✓ Xử lý exception để tránh chương trình bị dừng đột ngột.
- ✓ Dùng collections framework để lưu trữ dữ liệu hiệu quả.

43. GÓI (PACKAGE) TRONG JAVA

Package dùng để nhóm các class và interface liên quan. Giúp tránh xung đột tên và kiểm soát truy cập.

Lợi ích

- Tránh xung đột tên.
- Quản lý code có tổ chức.
- Cung cấp bảo vệ truy cập.
- Dễ bảo trì và tái sử dụng.

```
// Tạo một package
package com.example.demo;
public class Hello {
    public void show() {
        System.out.println("Hello from package");
    }
}
```

45. CLASS STRING

String là một chuỗi ký tự. Class String là **immutable** (không thể thay đổi sau khi tạo).

Method	Mô tả
length()	Trả về độ dài chuỗi
charAt(index)	Trả về ký tự tại index
substring(a,b)	Trả về chuỗi con (a đến b-1)
toUpperCase()	Chuyển thành chữ hoa
equalsIgnoreCase(s)	Số sánh không phân biệt hoa/thường
trim()	Xóa khoảng trắng đầu/cuối
indexOf(ch/str)	Trả về vị trí xuất hiện đầu tiên

Vi dụ:

```
String s = " Hello Java ";
System.out.println(s.length()); // 13
System.out.println(s.trim()); // "Hello Java"
System.out.println(s.toUpperCase()); // HELLO JAVA
```

47. ARRAYLIST TRONG JAVA

ArrayList là một mảng có kích thước tự thay đổi được (resizable). Nó implement interface List.

Đặc điểm

- Giữ thứ tự thêm vào (insertion order).
- Cho phép phần tử trùng lặp.
- Truy cập theo index, kích thước động.

```
ArrayList<Integer> list = new ArrayList<>();
list.add(10); list.add(20); list.add(30);
list.add(1, 15); // Chèn 15 tại index 1
System.out.println(list); // [10, 15, 20, 30]
```

44. CÂU LỆNH IMPORT

Import dùng để import class và interface từ package khác.

Cú pháp	Mô tả
import package.Class;	Import 1 class cụ thể
import package.*;	Import mọi class/interface trong package
import static pkg.Class.*;	Import mọi thành viên static của class

Vi dụ:

```
// import 1 class cụ thể
import java.util.ArrayList;
// import tất cả các class
import java.util.*;
// import thành viên static
import static java.lang.Math.*;
```

46. STRINGBUILDER & STRINGBUFFER

Dùng để tạo chuỗi có thể thay đổi (mutable).

StringBuilder (non-sync)	StringBuffer (sync)
Nhanh hơn, không thread-safe	Chậm hơn, thread-safe
Dùng trong môi trường single-thread	Dùng trong môi trường multi-thread

Method chung (giống nhau ở cả 2):

append(data) : Thêm vào cuối
insert(index, data) : Chèn tại vị trí
reverse() : Đảo ngược chuỗi

Vi dụ:

```
StringBuilder sb = new StringBuilder("Java");
sb.append(" Programming"); // Java Programming
sb.reverse(); // repeLevoD avaJ
```

48. WRAPPER CLASS

Wrapper class dùng để "bọc" kiểu dữ liệu primitive thành object.

Primitive	Wrapper Class	Vi dụ
byte	Byte	Byte b=10;
short	Short	Short s=100;
int	Integer	Integer i=1000;
long	Long	Long l=1000000L;
float	Float	Float f=10.5f;
double	Double	Double d=10.55;

📌 **Ghi nhớ:** String, StringBuilder và ArrayList là những công cụ dùng hàng ngày khi lập trình Java thực tế. 😊

49. FINAL, FINALLY, FINALIZE

Ba từ khác nhau về ý nghĩa và cách dùng.

Từ khóa	Ý nghĩa	Dùng với
final	Hạn chế thay đổi.	biến, method, class
finally	Block luôn thực thi.	try-catch
finalize()	Method GC gọi trước khi hủy object.	method của Object class

Ví dụ (final variable):

```
class Test {  
    final int MAX = 100; // không đổi được  
    void show() {  
        System.out.println("MAX = " + MAX);  
    }  
}
```

Lưu ý: Tránh dùng finalize(). Nó đã deprecated (không khuyến khích dùng).

51. STATIC TRONG JAVA

static thuộc về class, không thuộc về object nào.

- Được chia sẻ bởi mọi object.
- Truy cập qua tên class.
- Bộ nhớ chỉ cấp phát 1 lần khi class được load.

```
class Counter {  
    static int count = 0; // dùng chung mọi object  
    Counter() { count++; }  
    static void showCount() {  
        System.out.println("Count = " + count);  
    }  
}
```

53. ENUM TRONG JAVA

Enum dùng để định nghĩa một tập hằng số cố định.

```
enum Day {  
    MONDAY, TUESDAY, WEDNESDAY,  
    THURSDAY, FRIDAY, SATURDAY, SUNDAY  
}  
Day d = Day.MONDAY;  
System.out.println("Day = " + d);
```

55. THAM SỐ DÒNG LỆNH (COMMAND LINE ARGUMENTS)

Tham số được truyền từ command line vào chương trình Java.

```
class Add {  
    public static void main(String[] args) {  
        int a = Integer.parseInt(args[0]);  
        int b = Integer.parseInt(args[1]);  
        System.out.println(a+b);  
    }  
}  
// Chạy: javac Add.java && java Add 10 20
```



Ghi nhớ: this dùng cho class hiện tại, super dùng cho class cha - đừng nhầm lẫn hai khái niệm này. 😊

50. THIS VS SUPER

Cả hai đều là biến tham chiếu.

	this	super
Tham chiếu tới	Instance class hiện tại	Instance class cha
Dùng để	Truy cập thành viên class hiện tại	Truy cập thành viên class cha
Constructor	this()	super()

Ví dụ:

```
class A { void show(){System.out.println("In A");} }  
class B extends A {  
    void show() {  
        super.show(); // gọi show() của cha  
        System.out.println("In B");  
    }  
}
```

52. NESTED CLASS (LỚP LỒNG NHAU)

Một class được khai báo bên trong một class khác.

Các loại:

- Static Nested Class
- Non-static (Inner) Class
- Local Inner Class (bên trong method)
- Anonymous Inner Class (không tên)

Ví dụ (Inner Class):

```
class Outer {  
    int x = 10;  
    class Inner {  
        void show(){System.out.println("x = "+x);}  
    }  
}  
// Cách dùng:  
Outer o = new Outer();  
Outer.Inner i = o.new Inner();  
i.show();
```

54. AUTO BOXING & UNBOXING

Tự động chuyển đổi kiểu primitive sang object và ngược lại.

```
int a = 20; // primitive  
Integer obj = a; // auto boxing  
int b = obj; // auto unboxing
```

56. JDK, JRE, JVM

Ba thành phần quan trọng trong môi trường Java.

Thành phần	Mô tả
JDK (Java Development Kit)	Cung cấp công cụ phát triển. Gồm JRE + compiler + tools.
JRE (Java Runtime Environment)	Cung cấp môi trường chạy chương trình Java.
JVM (Java Virtual Machine)	Thực thi Java bytecode, đảm bảo độc lập nền tảng.

57. ĐẶC ĐIỂM CỦA JAVA

Java được thiết kế để đơn giản, robust và độc lập nền tảng.

- Simple và dễ học
- Object-Oriented
- Platform Independent (WORA)
- Secure
- Robust
- Architecture Neutral
- Portable
- High Performance
- Multithreaded
- Distributed

Ví dụ (Platform Independent):

```
// Code Java chạy được trên mọi nền tảng
class Hello {
    public static void main(String[] args) {
        System.out.println("Hello Java");
    }
}
```

58. JAVA VS C++

Cả hai đều là ngôn ngữ OOP, nhưng Java đơn giản và bảo mật hơn.

Đặc điểm	Java	C++
Độc lập nền tảng	Có (WORA)	Không
Pointer	Không có pointer tường minh	Có
Đa kế thừa	Không (qua class)	Có
Nạp chồng toán tử	Có	Có
Quản lý bộ nhớ	Tự động (GC)	Thủ công (new/delete)
Preprocessor	Không	Có
Kế thừa	Đơn (class), đa (interface)	Đa

59. VÙNG NHỚ TRONG JAVA

Java quản lý bộ nhớ tự động.

Vùng nhớ	Mô tả
Method Area	Lưu cấu trúc class, biến static, code method.
Heap Area	Lưu object và biến instance.
Stack Area	Lưu lời gọi method và biến local (theo LIFO).
PC Register	Lưu địa chỉ lệnh đang thực thi.
Native Method Stack	Dùng cho native method (code C/C++).

Method Area Heap Area Stack Area PC Register Native Method Stack

Được quản lý bởi JVM

60. KIỂU DỮ LIỆU TRONG JAVA

Java là ngôn ngữ statically typed.

(A) Kiểu Primitive

Kiểu	Kích thước	Mô tả	Giá trị mặc định
byte	1 byte	Số nguyên -128 đến 127	0
int	4 bytes	Số nguyên	0
float	4 bytes	Số thập phân	0.0f
char	2 bytes	1 ký tự (Unicode)	'\u0000'
boolean	1 bit	true hoặc false	false

(B) Kiểu Non-Primitive (Reference)

String, Array, Class, Interface, Object, Wrapper Class, Collection, v.v.

61. CHUYỂN ĐỔI KIỂU (TYPE CONVERSION)

Hai loại: Implicit (tự động) và Explicit (thủ công).

(A) Implicit (Widening)

byte → short → int → long → float → double

(B) Explicit (Narrowing)

double → float → long → int → short → byte

⚠️ Có thể mất dữ liệu khi narrowing.

63. BIẾN TRONG JAVA

Variable là tên đặt cho một vị trí bộ nhớ.

Quy tắc đặt tên

- Phải bắt đầu bằng chữ cái, \$ hoặc _.
- Không được bắt đầu bằng số.

62. LITERAL TRONG JAVA

Literal là các giá trị cố định.

- Integer Literal: int a = 10;
- Floating Literal: float f = 10.5f;
- Character Literal: char ch = 'A';
- String Literal: String s = "Hello";
- Boolean Literal: boolean b = true;
- Null Literal: String s = null;
- Long Literal: long l = 100L;
- Hexadecimal Literal: int x = 0x1A;
- Octal Literal: int y = 012;
- Binary Literal (Java 7+): int z = 0b1010;

64. KEYWORD VÀ IDENTIFIER

Keyword là từ để dành. Identifier là tên do người dùng đặt.

(A) Một số Keyword trong Java

```
abstract boolean break byte case catch char
class const continue default double do else
extends false final finally float for if
implements import instanceof int interface long native
```



Ghi nhớ: Nhớ chắc kiểu dữ liệu và cách chuyển đổi giúp tránh rất nhiều lỗi runtime khó chịu.



65. TOÁN TỬ TRONG JAVA

Operator là ký hiệu thực hiện phép toán trên biến và giá trị.

Loại	Toán tử	Ví dụ
Arithmetic	+ - * / %	a+b, a%b
Relational	= < > <= >=	a > b
Logical	&& !	(a>b)&&(c>d)
Assignment	= += -= *= /= %=	a += 5
Unary	++ -- + - !	i++, --j
Ternary	cond ? true : false	(a>b)?a:b
Bitwise	& ^ ~ << >> >>>	a&b, a<<1

Ví dụ:

```
int a=10, b=3;
int sum = a+b;    // 13
int rem = a%b;    // 1
int max = (a>b)?a:b; // 10
```

66. CÂU LỆNH ĐIỀU KHIỂN

Control statement điều khiển luồng thực thi.

Loại	Mô tả
if / if-else	Thực thi block khi điều kiện đúng
else-if ladder	Kiểm tra nhiều điều kiện tuần tự
switch	Chọn 1 block trong nhiều lựa chọn
for / while	Lặp khi biết/không biết trước số lần
do-while	Thực thi ít nhất 1 lần
break / continue	Dừng / bỏ qua vòng lặp hiện tại

Ví dụ (for loop):

```
for (int i=1; i<=5; i++) {
    System.out.print(i+" "); // 1 2 3 4 5
}
```

67. MẢNG TRONG JAVA

Array là tập hợp có kích thước cố định của các phần tử cùng kiểu.

```
int[] arr = {10, 20, 30, 40, 50};
System.out.println(arr[0]); // 10
arr[2] = 99;
for (int i=0; i<arr.length; i++) {
    System.out.print(arr[i]+" "); // 10 20 99 40 50
}
```

Mảng 1 chiều

```
int[] a = new int[5];
```

Mảng 2 chiều

```
int[][] b = new int[2][3];
```

68. PHƯƠNG THỨC TRONG JAVA

Method là khối code thực hiện một tác vụ cụ thể.

```
int add(int a, int b) { return a+b; }
public static void main(String[] args) {
    int sum = add(5, 3);
    System.out.println("Sum = "+sum); // 8
}
```

Lợi ích

- Tái sử dụng code
- Dễ đọc hơn
- Dễ bảo trì

Các loại Method

- Built-in (VD: length())
- Do người dùng định nghĩa

69. NẠP CHỒNG PHƯƠNG THỨC

Cùng tên method nhưng khác tham số (kiểu/số lượng/thứ tự).

```
int add(int a,int b){return a+b;}
double add(double a,double b){return a+b;}
// System.out.println(add(2,3)); // 5
// System.out.println(add(2.5,3.5)); // 6.0
```

☆ **Quy tắc:** Phải khác nhau ở tham số. Chỉ khác kiểu trả về là chưa đủ.

71. TỪ KHÓA THIS

this tham chiếu tới object hiện tại.

```
class Person {
    int age;
    Person(int age) { this.age = age; }
}
```

70. CONSTRUCTOR TRONG JAVA

Constructor khởi tạo object. Tên trùng class, không có kiểu trả về.

```
class Student {
    int id; String name;
    Student(int id,String name){ // parameterized
        this.id=id; this.name=name;
    }
    Student(){ this.id=0; this.name="Unknown"; } // default
}
```

72. GARBAGE COLLECTION

Là quá trình thu hồi bộ nhớ bằng cách xóa các object không còn dùng đến.

- JVM tự động phát hiện object không dùng.
- Thu hồi bộ nhớ và làm cho nó sẵn sàng sử dụng lại.
- Lập trình viên không thể gọi GC trực tiếp.

⚡ **Ghi nhớ:** Method overloading và constructor giúp code linh hoạt hơn khi khởi tạo và tái sử dụng. 😊

75. STRING TRONG JAVA (QUAN TRỌNG)

String là một object đại diện cho chuỗi ký tự.

Cách tạo String

- Bằng string literal: String s1 = "Hello";
- Bằng từ khóa new: String s2 = new String("Hello");
- Bằng mảng ký tự: char[] ch={'H','i'}; String s3=new String(ch);
- Bằng substring: String s4 = s1.substring(1,4);

Method thường dùng

length()	trả về độ dài	equalsIgnoreCase(s)	bỏ qua hoa/thường
charAt(i)	ký tự tại index i	concat(s)	nối chuỗi
substring(i,j)	chuỗi con	toUpperCase()	chuyển hoa
equals(s)	so sánh nội dung	trim()	xóa khoảng trắng

💡 String là **immutable**. Sau khi tạo, không thể thay đổi.

76. STRING POOL TRONG JAVA

String pool là vùng nhớ đặc biệt trong heap.

Quy tắc

- String literal được lưu trong string pool.
- Nếu literal đã tồn tại, trả về cùng tham chiếu.
- Từ khóa new luôn tạo object mới trong heap.

String Pool (trong Heap)



Ví dụ:

```
String s1 = "Hello";
String s2 = "Hello";
String s3 = new String("Hello");
System.out.println(s1 == s2); // true
System.out.println(s1 == s3); // false
```

77. TỪ KHÓA FINAL

final dùng để hạn chế thay đổi.

Dùng cho	Có thể áp dụng cho	Hiệu ứng
Final Variable	Biến	Không đổi được giá trị
Final Method	Method	Không thể override
Final Class	Class	Không thể kế thừa

79. SINGLETON CLASS

Singleton class chỉ có duy nhất một object.

Các bước:

- Đặt constructor là private.
- Tạo object static.
- Cung cấp method public static để lấy object.

```
class Singleton {
    private static Singleton obj;
    private Singleton() {}
    public static Singleton getInstance(){
        if(obj==null) obj=new Singleton();
        return obj;
    }
}
```

78. TỪ KHÓA STATIC

static thuộc về class, không thuộc về object.

Cách dùng của static

- Biến static (biến của class)
- Method static
- Block static

```
class Counter {
    static int count = 0; // dùng chung mọi object
    Counter() { count++; }
}
```

80. INNER CLASS (LỚP BÊN TRONG)

Class được khai báo bên trong một class khác.

Loại	Truy cập
Member Inner Class	Truy cập mọi thành viên class ngoài
Static Nested Class	Chỉ truy cập thành viên static
Local Inner Class	Truy cập biến final/effectively final
Anonymous Inner Class	Dùng với interface hoặc abstract class

81. PACKAGE TRONG JAVA

Package là nhóm các class và interface liên quan.

```
// File: com/example/demo/Hello.java
package com.example.demo;
public class Hello {
    public void show(){ System.out.println("Hello"); }
}
```

82. ACCESS MODIFIER

Kiểm soát phạm vi truy cập của class, method và biến.

Modifier	Cùng Class	Cùng Package	Subclass	Toàn cục
private	✓	✗	✗	✗
default	✓	✓	✗	✗

💡 **Ghi nhớ:** String immutable + string pool là chủ đề hay gặp nhất trong phỏng vấn Java - nắm thật chắc phần này. 😊

83. EXCEPTION TRONG JAVA

Exception là sự kiện ngoài ý muốn làm gián đoạn luồng chạy bình thường.

Loại	Checked (Compile Time)	Unchecked (Runtime)
Kiểm tra bởi	Compiler	JVM
Ví dụ	IOException, SQLException	NullPointerException, ArithmeticException
Xử lý	Bắt buộc phải xử lý	Không bắt buộc

Ví dụ:

```
try {
    int a=10, b=0;
    int c = a/b; // Exception
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
} finally { System.out.println("Finally block"); }
```

84. COLLECTION FRAMEWORK

Cung cấp kiến trúc để lưu trữ và thao tác với nhóm object.



Interface phổ biến

List: có thứ tự, cho phép trùng lặp

Set: không thứ tự, không trùng lặp

Queue: xử lý phần tử theo trình tự

Map: cặp Key-Value (không thuộc Collection)

85. GENERICS TRONG JAVA

Cho phép object type-safe, loại bỏ việc ép kiểu thủ công.

Lợi ích

- Kiểm tra kiểu lúc compile-time
- Loại bỏ ClassCastException
- Tái sử dụng code

```
ArrayList<String> list = new ArrayList<>();
list.add("Java"); list.add("Awesome");
String s = list.get(0); // không cần ép kiểu
```

86. ENUM TRONG JAVA

Enum dùng để định nghĩa tập hằng số cố định.

```
enum Day { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY }
Day d = Day.MONDAY;
System.out.println(d); // MONDAY
```

87. XỬ LÝ FILE TRONG JAVA

Java cung cấp các class để đọc/ghi dữ liệu vào file.

Class quan trọng

- File - Đại diện file hoặc thư mục.
- FileReader/FileWriter - Character stream.
- FileInputStream/FileOutputStream - Byte stream.

Ví dụ (đọc file):

```
FileReader fr = new FileReader("input.txt");
BufferedReader br = new BufferedReader(fr);
String line;
while((line=br.readLine())!=null){ System.out.println(line); }
br.close(); fr.close();
```

⚠ Luôn đóng stream để giải phóng tài nguyên.

88. MULTITHREADING TRONG JAVA

Cho phép thực thi đồng thời nhiều phần của chương trình.

```
class MyThread implements Runnable {
    public void run() {
        for(int i=1; i<=5; i++) System.out.println("Thread: "+i);
    }
}
Thread t = new Thread(new MyThread());
t.start();
```

89. JAVA I/O STREAM

Stream dùng để đọc/ghi dữ liệu.

Loại	Mục đích	Ví dụ Class
Byte Stream	Xử lý dữ liệu 8-bit	FileInputStream
Character Stream	Xử lý dữ liệu 16-bit	FileReader
Buffered Stream	Cải thiện hiệu năng	BufferedReader

90. NGÀY GIỜ TRONG JAVA (API MỚI)

Package java.time (từ Java 8) cung cấp API date-time hiện đại.

Class quan trọng

- LocalDate - biểu diễn ngày (yyyy-mm-dd)
- LocalTime - biểu diễn giờ (hh:mm:ss)
- LocalDateTime - biểu diễn cả ngày và giờ
- DateTimeFormatter - định dạng ngày giờ

```
LocalDate date = LocalDate.now();
```



Ghi nhớ: Collection framework và multithreading là hai chủ đề nâng cao rất hay được hỏi khi phỏng vấn.



91. ĐẶC ĐIỂM JAVA 17+ (TỔNG QUAN)

Các tính năng hiện đại giúp Java tốt hơn.

Tính năng	Mô tả
Records	Cú pháp gọn cho dữ liệu bất biến (immutable)
Switch Expressions	Switch gọn hơn với <code>→</code> và giá trị trả về
Text Blocks	Chuỗi nhiều dòng đúng dấu ba nháy kép
var (Local Variable Type Inference)	Để compiler tự suy ra kiểu dữ liệu
Sealed Classes	Giới hạn class nào được kế thừa/implement
Pattern Matching cho instanceof	Ít code thừa hơn, type-safe hơn

Ví dụ (Text Block):

```
String json = """
{
  "name": "Nikhil",
  "role": "Developer"
}
""";
```

92. JAVADOC COMMENT

Dùng để tạo tài liệu (documentation) từ source code.

```
/**
 * Method này cộng hai số.
 * @param a số thứ nhất
 * @param b số thứ hai
 * @return tổng của a và b
 */
public int add(int a, int b) { return a+b; }
```

Các tag thường dùng

@param tham số | @return giá trị trả về | @author tác giả | @throws exception | @see tham chiếu

Tạo tài liệu:

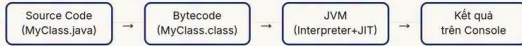
```
javadoc -d docs MyClass.java
```

93. CƠ BẢN VỀ COMMAND LINE

Các lệnh hữu ích khi phát triển Java.

Lệnh	Mục đích
<code>javac File.java</code>	Biên dịch file Java
<code>java File</code>	Chạy class đã biên dịch
<code>javap ClassName</code>	Dịch ngược class (xem bytecode)
<code>java -version</code>	Kiểm tra phiên bản Java

94. LƯỚING THỰC THI CHƯƠNG TRÌNH JAVA



95. LỖI THƯỜNG GẶP (CHO NGƯỜI MỚI)

Lỗi	Nguyên nhân
Syntax error	Sai cấu trúc code
Runtime error	Lỗi khi chương trình đang chạy
NullPointerException	Dùng tham chiếu null
ArrayIndexOutOfBoundsException...	Truy cập index không hợp lệ

⚠ Đọc kỹ thông báo lỗi - nó sẽ cho bạn biết câu trả lời.

96. BEST PRACTICES

- ✓ Tuân theo quy tắc đặt tên (camelCase cho biến)
- ✓ Giữ method ngắn gọn, tập trung 1 việc
- ✓ Viết comment và Javadoc có ý nghĩa
- ✓ Xử lý exception đúng cách
- ✓ Tránh hard-code giá trị (dùng hằng số)
- ✓ Đặt tên biến/method rõ nghĩa

97. CÂU HỎI PHÒNG VẤN NHANH

Q1. Java là gì?

Java là ngôn ngữ lập trình high-level, object-oriented, độc lập nền tảng.

Q2. JVM là gì?

JVM là engine chạy Java bytecode và cung cấp khả năng độc lập nền tảng.

Q3. Khác nhau giữa JDK, JRE và JVM?

Thành phần	Bao gồm
JDK	JRE + công cụ phát triển (compiler, debugger, v.v.)

98. TỔNG KẾT

Những điểm chính cần nhớ về Introduction to Java.

- 🔥 Java đơn giản, bảo mật và độc lập nền tảng.
- 🔥 Mọi thứ trong Java là class và object.
- 🔥 JVM giúp Java portable trên mọi nền tảng.
- 🔥 Kiến thức cơ bản là nền tảng cho các chủ đề nâng cao.
- 🔥 Luyện tập chương trình nhỏ mỗi ngày.
- 🔥 Hiểu sâu kiến thức cơ bản trước khi tiến xa hơn.

❤ Kiên trì, tò mò và không ngừng xây dựng kỹ năng của bạn. Bạn làm được mà!



Cảm ơn bạn đã theo dõi hết cẩm nang! Chúc bạn học Java thật vui và code thật nhiều.

